

VIRTUAL MEMORY

A first important step in memory management was to stop holding on to the idea that a process (image) should be stored as a single contiguous block in memory, which gave rise to paging and segmentation. The next step could be indicated by the realisation that possibly not all pages belonging to a single process have to be present in memory at the same time. We already saw in the section on caches that there is such a thing as the locality of reference principle. In the context of paging, this same principle suggests that only a limited number of pages may be needed over short periods of time. In short, without sacrificing speed, one could store a part of the pages of a process in secondary memory (swap space). When a page is swapped, one can use the corresponding page table entry to indicate where the page can be found (on the I/O device).

The idea that not all pages need to be in memory at the same time not only allows us to (partially) load more processes into memory, but a single process can even occupy a larger address space than the one physically present. That's why we call the logical addresses, which are created by the compiler, virtual addresses.

In this chapter, we focus on paging systems. The situation when using virtual memory shows some similarities with the caching issues from the previous chapters. The pages of a process are only partially present in memory, which allows for faster access, the remaining pages are found in swap space. When a page is present in memory, we can immediately access it, otherwise it must be loaded. Instead of a *cache miss*, if a page is not present, we speak of a *page fault*. The handling of page faults is the responsibility of the operating system. In order to know which page caused the page fault, the paging hardware stores this page number in a register. An important difference with caching is that the search for pages is done through a page table and not through a direct/associative mapping.

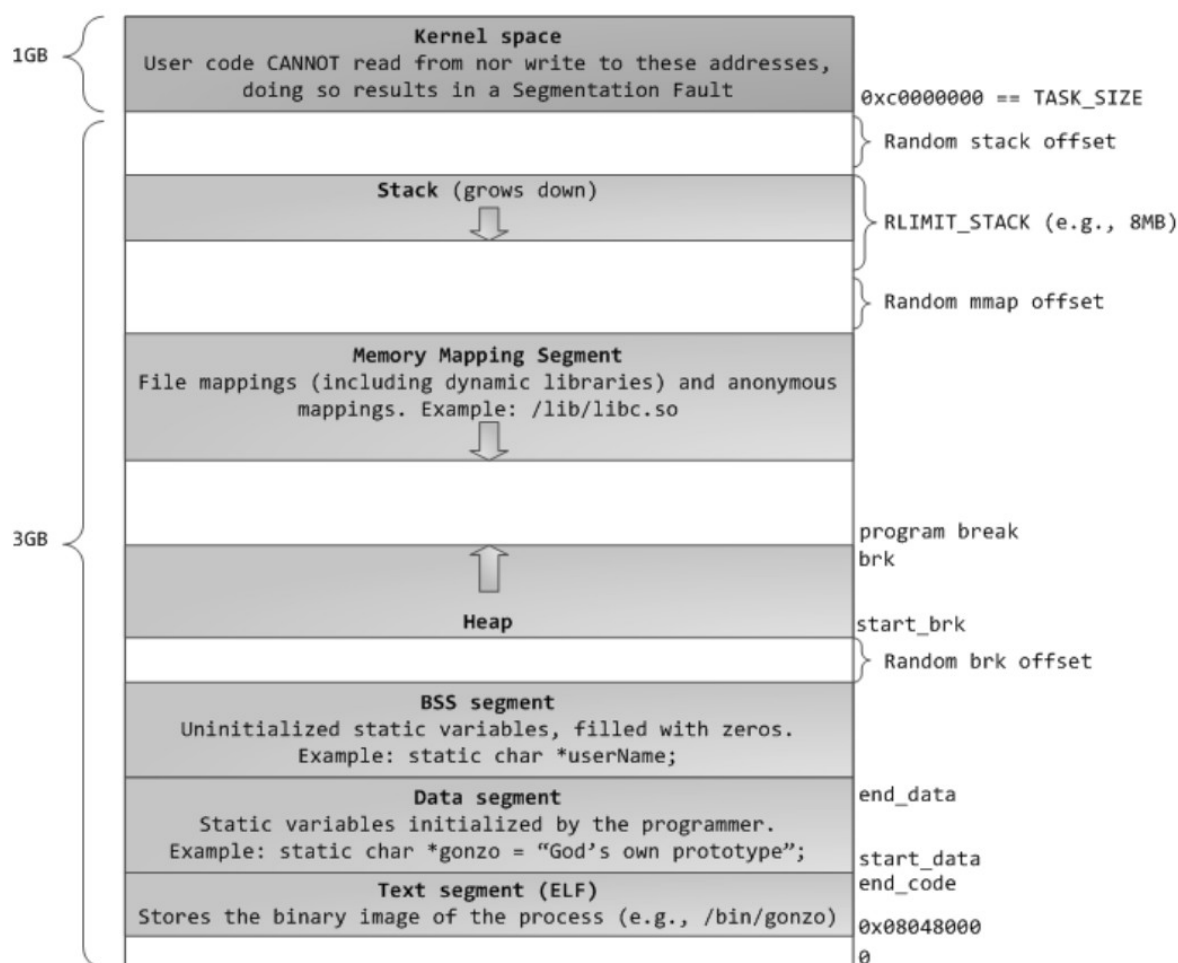


Fig. 28: Virtual memory layout in Linux

1 Virtual memory layout

Before looking in more detail at how to implement page tables without consuming too much memory, let's have a look at how a process typically divides up the virtual memory. We look at a typical memory layout in Linux on a 32 bit machine. Figure 28 shows that the virtual address space of a process is divided into several segments. These segments have nothing to do with the segmentation discussed in Section 3 of the previous chapter.

We see that the top GByte (in Windows default 2 Gbyte) is reserved for mapping the memory used by the kernel. This part of the mapping is present in the page table of each process and is also identical for each process (which ensures that the kernel code can always function correctly). In other words, only the mapping of the lower 3 Gbyte has to be changed when changing processes. In the user part we see a stack, memory mapped, heap, two data and a text segment.

The stack segment is used to implement the user stack and has a maximum size (e.g. 8Mbyte). The amount of memory allocated to the stack will only increase during the execution of a process, while the content on the stack can of course shrink as well as grow. By itself 8Mbyte seems very little for all local variables and parameters, but in a lot of cases references to the actual data are kept on the stack and can be found in the heap segment.

The memory mapped segment is often used to load files into the memory for quick access to these files. This segment is used, among other things, to get fast access to *dynamic libraries* (e.g. .dll files in Windows or so files in Linux). These files form a library with functions that can be used by multiple applications. The heap segment is used for memory that is allocated during the execution of a program (e.g. via the C function malloc). Managing the memory that is part of the heap is often very complex. The remaining 3 segments are used to load the code and the global variables of the process, the data segment holds the values of the global variables already assigned at the start of the process and the BSS segment provides the necessary memory for the remaining global variables.

The random offsets shown in Figure 28 do not contain any data and are only there to make the *hacker's* life a little more difficult. These fields ensure that the exact start position of the different segments is not known, which makes it a bit harder to *hack* a user program. When no random offsets are used and there is no limit on the size of the stack, you get the classic Linux memory layout shown in Figure 29. The common *segmentation fault* refers to the fact that a process tries to do something with respect to the above segments that is not allowed, such as a write operation to the text segment, a read to the kernel segment when the process is in user mode, causing a stack overflow, etc.

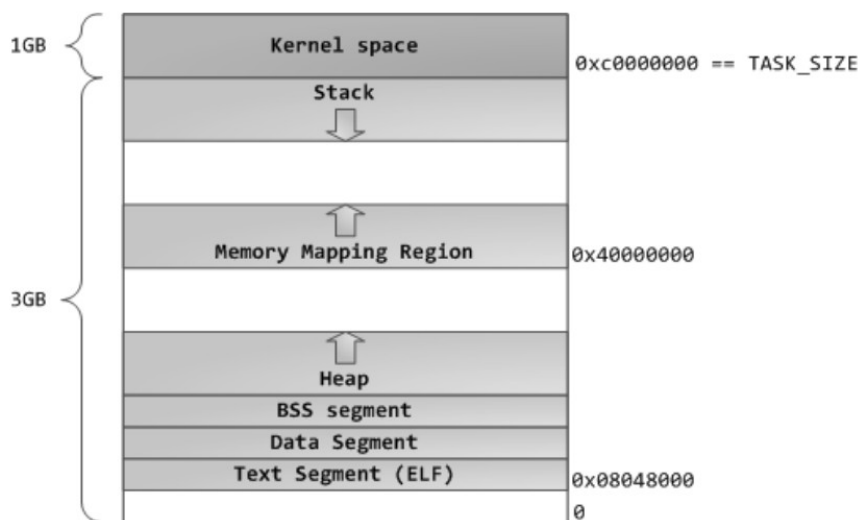


Fig. 29: Classic virtual memory layout in Linux

2 Page table organization

It would be desirable that the page table of a process is always present in memory (when the process is active). Finally, the processor must frequently perform an address translation (where a virtual page number is converted into a frame number) and it is important that the frame number can be found quickly. Ideally the start address of the page table is stored in a register of the processor, then the virtual page number is added to arrive at the page table entry that contains the frame number. This frame number together with the offset from the virtual address, the physical address (see Figure 30).

However, even when the virtual address space is not very big (e.g. 4 GByte) and the pages not too small (e.g. 4096 bytes) the page table contains very many entries (106 in our example). One could argue that many processes are much smaller and therefore do not require a large page table, but as we indicated earlier the different segments that make up the virtual address space are very spread out over this space. This means that even processes that require little memory cannot use a small page table.

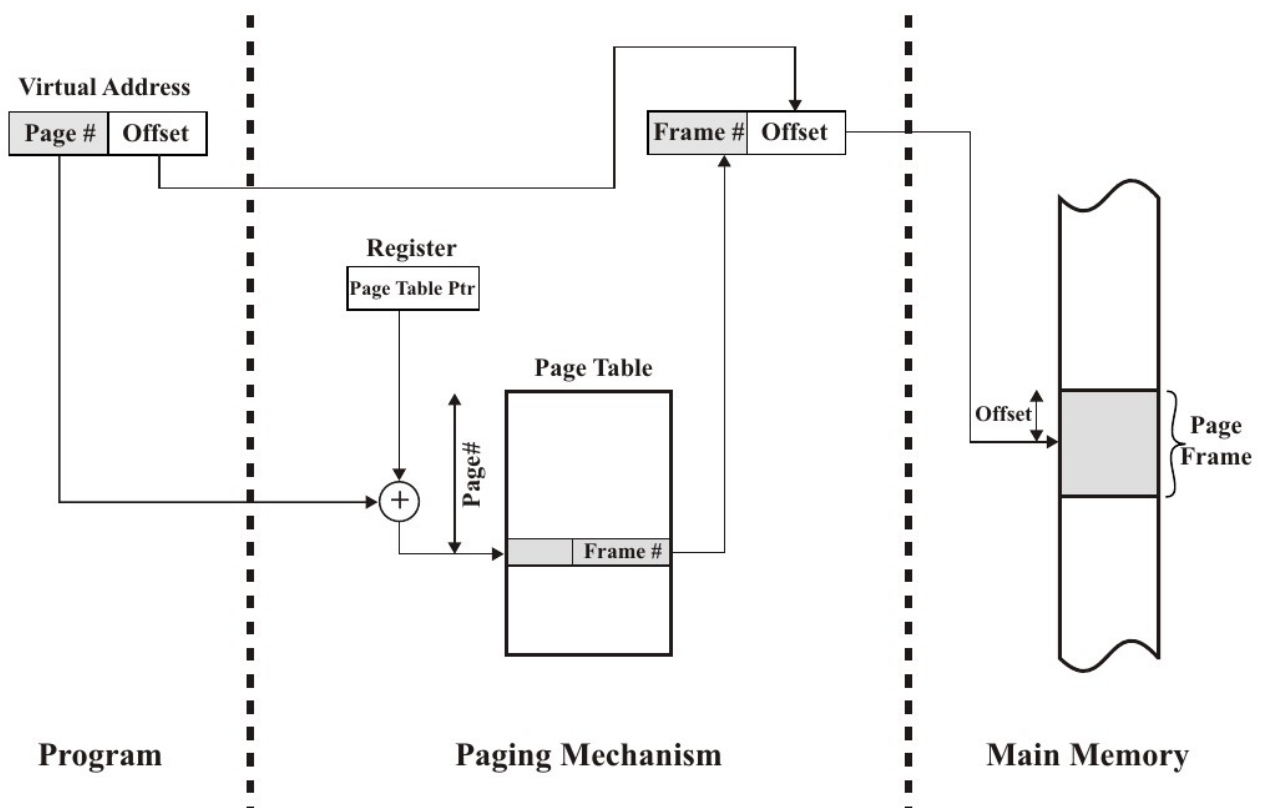


Fig. 30: Address translation in a paging system [Stallings 03].

2.1 Multilevel page tables

An often used solution to reduce the size of the page table is the *multilevel* page tables. In case of two levels the first n bits of the virtual page number are used as index to the level 1 page table (the so-called *page directory*). Each entry in this table refers again to a page table containing 2^m page entries and where we find the frame number. With this organization one always stores the level 1 page table in memory (since it is much smaller) and only loads the used level 2 tables. Because of this, unused memory areas will not be consumed anymore.

A logical and often-used choice for the number of entries in a level 2 page table is to choose it so that a level 2 page table is exactly one page in size and mapped to a single frame (e.g. x86 architectures). In this case, the following scenarios can occur when translating a virtual address, depending on whether the required level 2 page table is present in memory or not.

- 1 It could be that the required level 2 page table is present in memory, which means that we will find a frame number in the level 1 page table entry. When we use the last m bits of the virtual page number as an index to this level 2 page table, we find the frame number there that, together with the offset of the virtual address, forms the physical address. If the requested memory word is not present in memory, a page fault occurs and the virtual page is loaded into memory and the level 2 page table entry is modified.
- 2 If the level 2 page table is not in memory, a page fault occurs and we must first load it into memory. After that, a second page fault may occur if the memory word is not present in physical memory. A disadvantage of multilevel page tables is that a single lookup of a frame number requires multiple memory calls. This is especially a disadvantage in systems that support more than 4 GB of memory because they often use a page table with 3 levels. The 32 bit x86 architectures use 2 levels, where the 20 bit page number is divided into two pieces of 10 bits. The disadvantage of the multiple memory calls is for the most part taken care of by the TLB cache that we will discuss in Section 2.3.

2.2 Inverted page tables

Keeping track of a page entry per possible virtual page (per process) seems a bit cumbersome as there are many more virtual pages than physical ones. The advantage of this approach is of course that we can look up the frame numbers in the table via *direct access*. When we only keep a table with an entry for each physical page (or frame), we get a much smaller table.

Looking up a frame number is now a lot less easy: we have to go through all frames to see if one of them corresponds with the requested page of a certain process. In principle, we could use *associative access* for this, but it is a very expensive solution. Sequential access is cheap but too slow. A binary search method is efficient, but requires a lot of effort to add entries to the table (since it must be sorted).

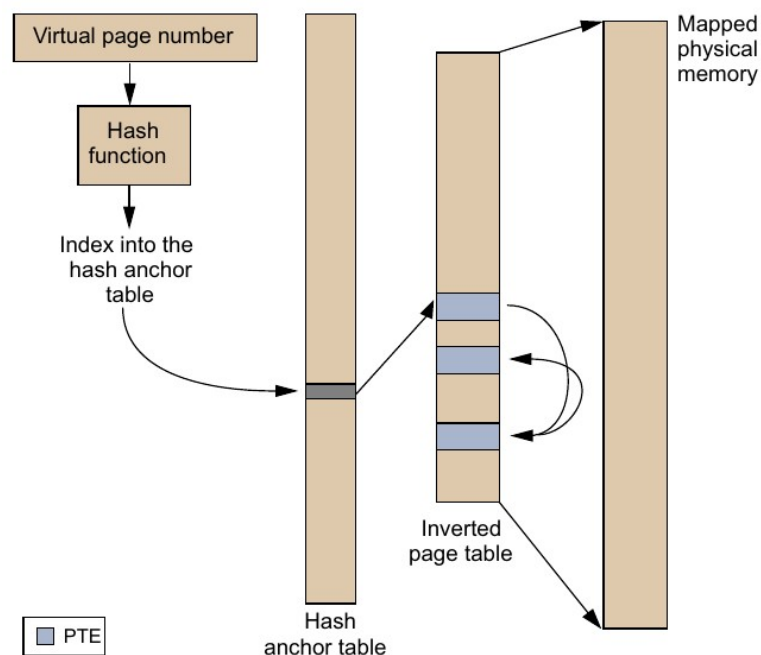


Fig. 31: Inverted page tables [Jacob 98].

The solution to these difficulties is *hashing*, a method used in many systems. The page/frame table is then extended with a hash anchor table (see Figure 31). When we want to find the frame number

of a certain page of a process, we will first run a hash function on this number (and the process id). This can easily be done by using a part of the bits as index to the *hash anchor table*. This table will in turn point to a certain entry in the page/frame table, whose *index*, with some luck, contains the frame number (if the page is in memory). Figure 32 shows this in more detail. The 2 bits in step two come from the fact that each byte is addressed and a page table entry is 4 bytes in size. The function of the TLB cache will become clear later.

We have to be a bit lucky as the hash function prints multiple process id and virtual page number pairs on the same table entry. When several of these page numbers are present in memory they form a linked list, which we call a *chain* (this can be done in the page table as shown in the figure or via a conflict resolution table). When the physical memory is composed of N frames, the number of possible outcomes M of the hash function will usually be slightly larger. By taking this value a bit larger we reduce the chance that there are pages on the same entry and thus make the chains a bit shorter. For certain tables, one can show that this approach usually yields a result after 1.5 entries in the page table (given that the page is in memory and with the necessary assumptions). Note that the hash anchor table allows us to choose $M > N$ without having to store the frame number explicitly in the page table entry (the index or offset indicates the number).

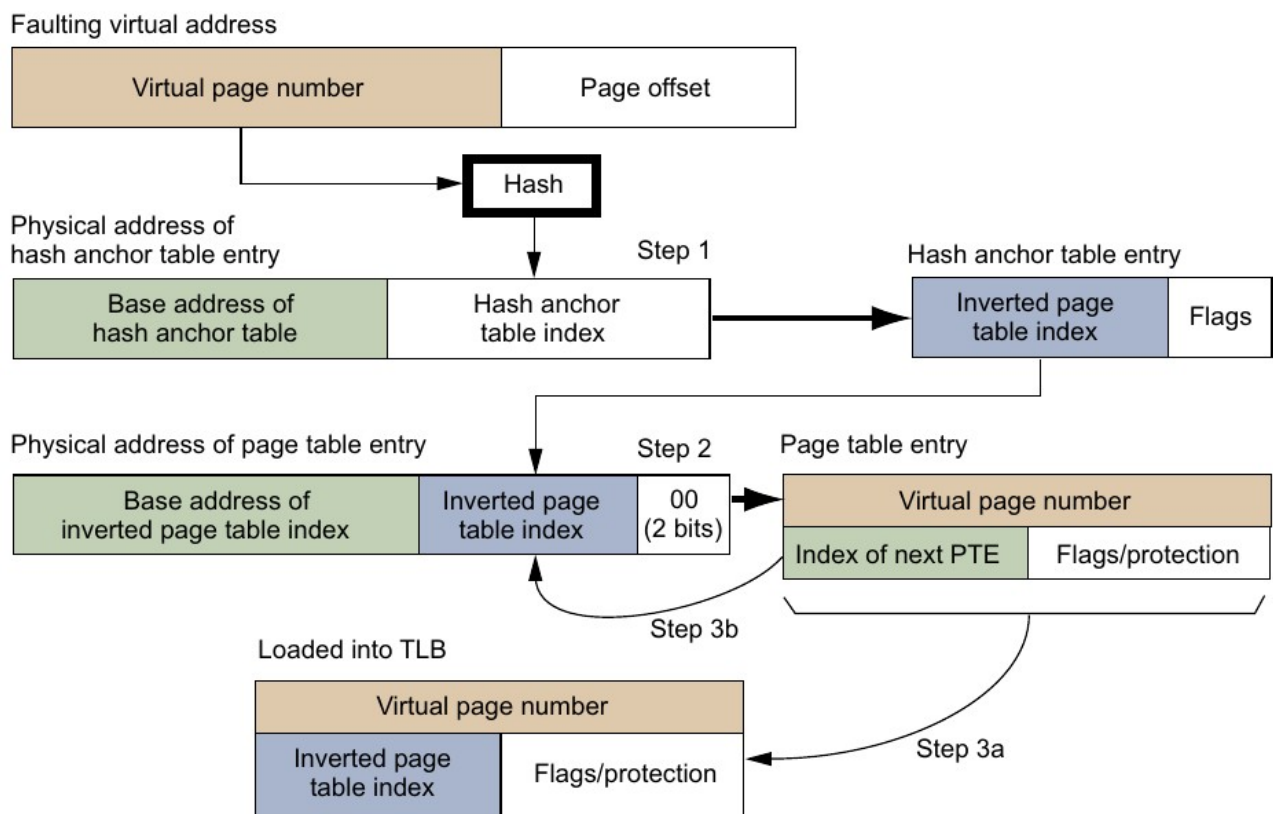


Fig. 32: Lookup algorithm for inverted page tables [Jacob 98].

Sharing pages is a problem when using inverted page tables. Each frame is only shown on one (or no) page.

2.3 Translation lookaside buffer

Despite the proposed data structures, translating virtual addresses into physical addresses remains a relatively slow operation. To ensure that this does not have to be done every time a virtual address appears, most systems are equipped with a *translation lookaside buffer* (TLB) cache. However, this cache does not contain user code or data, but a (small) part of the page table entries. Based on the reference of locality principle, this cache will quickly make the most recently encountered page table entries available to the processor. When the same translation is made several times in quick

succession, the first translation will be made using the page table and the result will be stored in the TLB cache. In subsequent translations the cache will be successfully consulted.

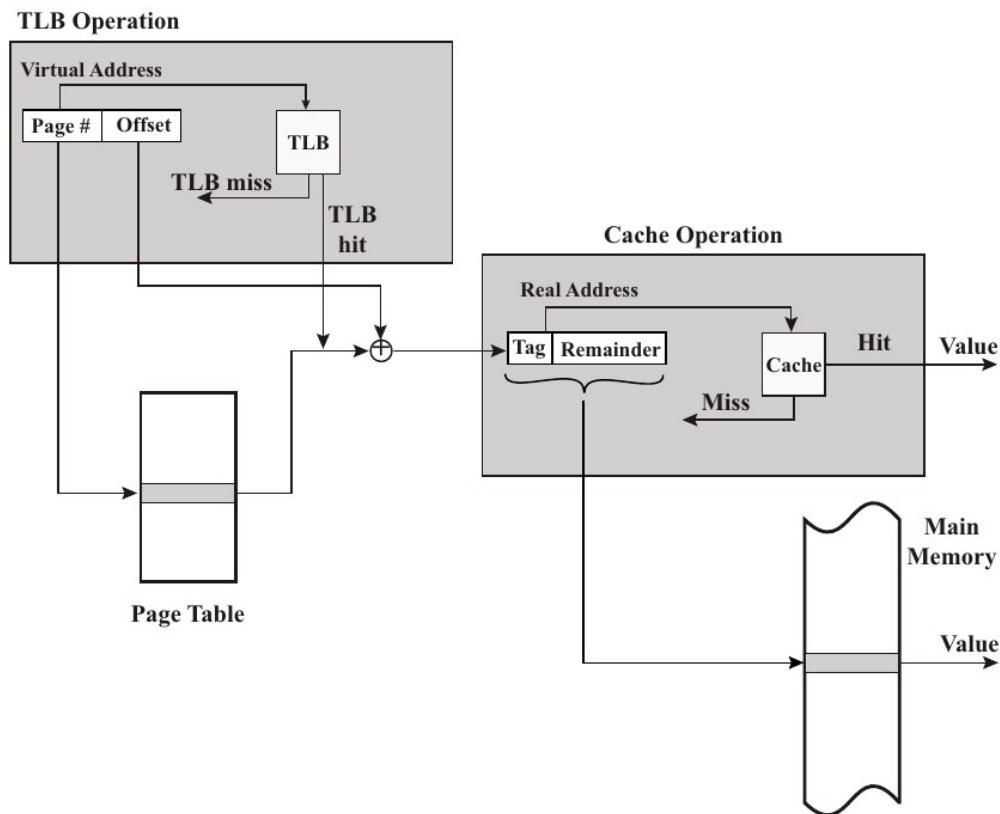


Fig. 33: TLB and cache operation [Stallings 03].

Looking up virtual page numbers in the cache can be done, as with any cache, in a number of ways: via direct, associative or set-associative access. The size of the tag depends on the chosen method. When a cache miss occurs, the processor will fall back on the page table in memory to perform the translation. The TLB cache usually consists of a limited number of registers that are part of the processor. For example, the 80486 architecture uses an associative cache that consists of 32 registers. The TLB cache also always contains page table entries that belong to the process currently in execution. This implies that every process change must also empty the TLB cache (this is called a *TLB flush*).

Retrieving a memory word in a computer system equipped with a data and TLB cache involves a number of steps (see Figure 33). First, the virtual page number of the word in the TLB cache will be looked up. If this results in a TLB miss, the page table in memory (or virtual memory) will be invoked. This in turn may result in a page fault, meaning that the operating system must first load the page containing the requested word. When the physical address is finally available to the processor, it looks it up in the data cache. If this results in a cache miss, the word is retrieved from memory. If a miss occurs in one of the caches, this cache will also be modified, using the cache replacement algorithm.

Like other caches, the TLB cache is sometimes divided into a data (D-TLB) and instruction (I-TLB) cache. The page table entries to pages with data and instructions are then stored in the D-TLB and I-TLB caches respectively.

3 Virtual memory: software support

When translating virtual addresses, everything is resolved by hardware, as long as no page fault

occurs. If a requested page is not in physical memory, the operating system must ensure that the page in question is loaded into memory. In doing so, it must make some important decisions.

First, the question arises whether we only load the requested page, i.e. *demand paging*, or suddenly load some extra pages, which is referred to as *prepaging*. Most systems use demand paging. Prepaging can be an interesting alternative when we keep the internal operation of harddisks and other storage media in mind.

A second more important decision is in which page frame to place the page to be loaded. This boils down to the determination of the page to be temporarily stored in secondary memory, as placement is not an issue in a paging system. This problem can be broken down into a number of subproblems which we will discuss separately: (i) how is the number of pages per process determined, (ii) can a page also occupy a frame of another process and (iii) once we have identified the set of candidate page frames, how do we choose the page to replace? The policy that answers the first two questions is called *resident set management*. The page replacement algorithm addresses the third question, which we will first examine more closely.

3.1 Page replacement algorithms

Notice that the page replacement algorithm has some similarities with the cache replacement algorithm. In both cases we have the same objectives: we want to replace the page/cache line to minimize page faults/cache misses. We see that a number of cache replacement algorithms recur in this section. Both problems are not equivalent in practice. For example, in the case of a set-associative mapping, the cache replacement algorithm will only have to choose from a very small set of candidate cache lines (only 2, 4 or 8), while the page replacement problem may select from a larger set.

Optimal algorithm: We start by discussing the optimal page (and also cache) replacement algorithm. This algorithm is guaranteed to result in the minimum number of page faults. However, it requires perfect knowledge of all future events, which makes it not implementable. This algorithm is therefore often used as a comparison or reference point. In the last section of this chapter, we will also formally prove the optimality of this algorithm. The optimal algorithm, often called the MIN algorithm, was developed by Belady in the 1960s (just like virtual memory) and consists of replacing all candidate pages with the page that will be requested furthest in the future.

Least Recently Used (LRU) algorithm: The LRU algorithm, which was already briefly discussed in cache replacement, will replace the least recently used page. It tries to mimic the behaviour of the MIN algorithm, without having the necessary future information. Thinking back to the locality of reference principle, this indeed seems a useful approach: recently requested pages are more likely to be requested again in the near future. For the implementation we can use a simple stack. Although the performance is usually very good, there are some serious drawbacks:

- Each time a page hit occurs, the requested page must be moved to the most recently used (MRU) position. Mutual exclusion for this position is therefore necessary when many threads/processes try to adjust the MRU position. For this purpose often a lock bit will be used which may lead to a strong performance degradation as all processes in turn only get access to the critical zone.
- For virtual memory, adjusting the MRU position for each cache hit requires a lot of processing power.
- LRU will easily replace pages that are requested frequently over short periods of time and then remain unused for a while. The frequency of a page is therefore of minor importance.

- The LRU performance is also sensitive to (long) sequences of pages that are requested only once (this is called a scan).

First In First Out (FIFO) algorithm: The FIFO algorithm replaces the page that has been in memory the longest. The idea is that this page may not have been used for a long time. Of course, this is often not the case, since we want to keep frequently requested pages in memory as long as possible. FIFO does have the advantage that the implementation consists of only one pointer and that cache hits do not require an update of the pointer. A curious side effect of FIFO is that, for certain page sequences, the presence of more memory can lead to more page faults. This effect is known as Belady's anomaly (see Figure 34).

Clock algorithm The Clock algorithm was introduced in 1968 by Frank Corbató. As algorithm, it is situated between FIFO and LRU. Besides a single *next frame* pointer, a single bit, the *use bit*, is kept per page frame. All frames are placed in a circular buffer. The use bit is set to 1 when loading a new page, as well as every time a page is hit. When there is a page fault, the next frame pointer will loop through the buffer until it finds a frame with a use bit set to 0. If the pointer points to a frame whose use bit is 1, it is set to 0 and the pointer moves on to the next frame.

The clock algorithm also requires relatively little hardware support. It suffices that the hardware supports a set of reference bits which indicate whether a certain page was requested. For each page hit, the hardware will set the corresponding bit to one (if it was not already equal to one). On a page miss, the clock algorithm will then use these reference bits.

Page Requests	3	2	1	0	3	2	4	3	2	1	0	4
Frame 1	3	3	3	0	0	0	4	4	4	4	4	4
Frame 2		2	2	2	3	3	3	3	3	1	1	1
Frame 3			1	1	1	2	2	2	2	2	0	0
Page Requests	3	2	1	0	3	2	4	3	2	1	0	4
Frame 1	3	3	3	3	3	3	4	4	4	4	0	0
Frame 2		2	2	2	2	2	2	3	3	3	3	4
Frame 3			1	1	1	1	1	1	2	2	2	2
Frame 4				0	0	0	0	0	0	1	1	1

(red indicates page fault)

Fig. 34: Belady's anomaly: with FIFO, more memory can lead to more page faults

Pages which are requested frequently can remain in memory for a long time without having to fall back on the LRU algorithm. An interesting variant of the Clock algorithm uses the *modified* or *update* bit which is kept for each page present in memory. First, the circular buffer is traversed to find a frame with both bits equal to 0. If we find such a frame, it is chosen. Otherwise, the buffer is rotated again to find a frame with the use bit set to 0, but with the modified bit set to 1. During this second rotation, the use bit is set to 0 if it was set to 1. If no frame is found after the second rotation, the first two steps/rotations are repeated. Because all use bits are set to zero in the meantime, this will always result in the selection of a frame. This variant tries to replace unchanged frames first, which is often more efficient, since the frame doesn't have to be written out first.

Meanwhile, numerous variants of the Clock algorithm have been developed, such as the use of a second pointer (see slides), and are used in numerous (contemporary) systems. In Figure 35, the evolution of each of the four previous algorithms is again demonstrated using an example.

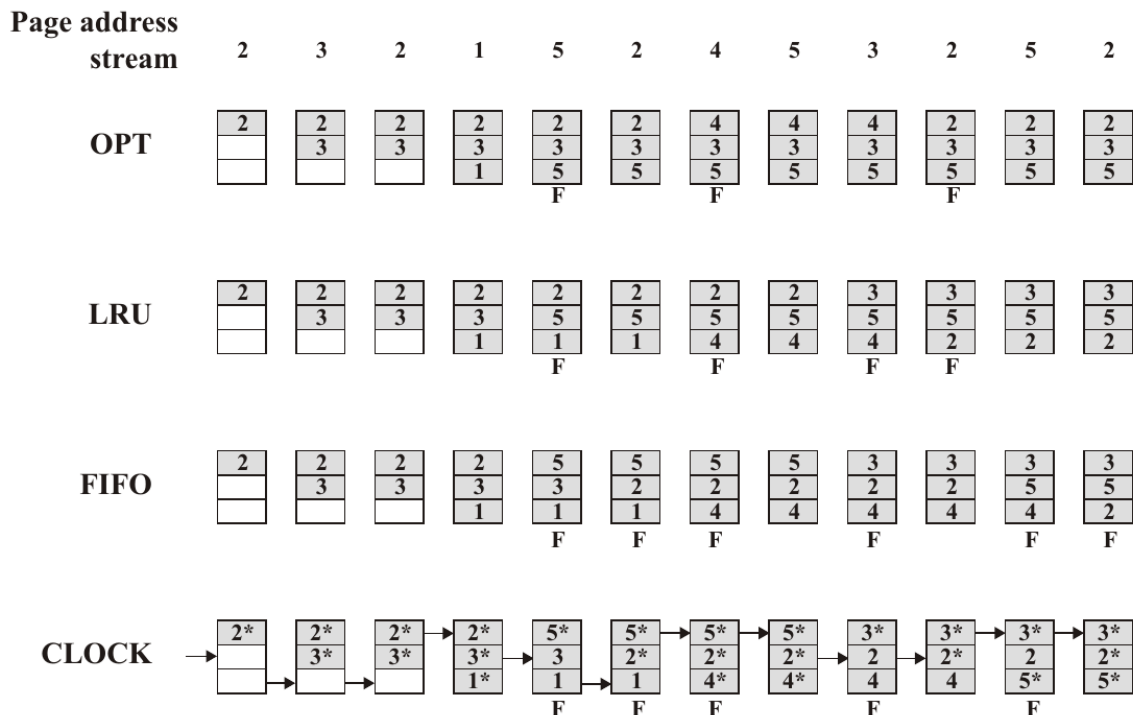


Fig. 35: Behavior of some page replacement algorithms [Stallings 03].

Page buffering and segmented FIFO: Page buffering is also a replacement algorithm that offers an intermediate solution for FIFO and LRU. We will explain it using the Segmented FIFO (SFIFO) algorithm published by Turner and Levy in 1981. SFIFO will keep two lists L1 and L2 of frames. When a new page is read into memory, it will be placed in front of the L1 buffer. This buffer follows a simple FIFO regime. The frame at the back of the L1 buffer is then added to the front of the L2 buffer. The frame that was at the end of the L2 buffer, will contain the new page. When a page hit occurs in the L1 buffer, nothing happens. However, the hit takes place in the L2 buffer, then this frame in the front of the L1 buffer added. Without L2 we get the FIFO system, if the L1 buffer contains only one frame then the algorithm is equivalent to LRU.

The strength of this algorithm is that the performance (for a good choice of the size of L1 and L2) is close to LRU, while the amount of work on a page hit is kept low. Pages that are requested frequently are referred to the L2 list from time to time, but if they are referred to again quickly enough, they end up in the L1 list again without ever being removed from memory. The L2 buffer, in a way, catches mistakes made by the FIFO algorithm.

This idea can also be worked out in combination with other replacement algorithms. This is called *page buffering*. In this context, the L2 list is often referred to as the free page list. The idea behind this is that the L2 list often also contains some frames that do belong to the *resident memory* of a process, these are the page frames that belong to a process, but which are not yet occupied by a page. A more advanced version of page buffering consists in keeping an L3 list. Pages that are removed from the L1 list will then go into the L2 or L3 buffer depending on whether they have been updated or not. Page buffering is also used as a safety net in the context of resident set management with variable allocation (see below).

3.2 Resident set management

When a page fault occurs, the replacement algorithm is activated to select a page frame from the candidate frames. Which frames are eligible is determined by the *resident set* policy. First of all, we must note that a part of the frames are never eligible in any case. These frames, *locked*, belong for instance to the kernel process. For the page frames which are not locked in memory, there are several possibilities.

A first distinction is made between systems that allocate a fixed or variable number of page frames per process (i.e., *fixed* or *variable* allocation). In addition, some systems will allow a page from one process to occupy a frame from another process. When pages can only claim their own page frames, we call it a local replacement policy, otherwise a global policy. An operating system which combines a local policy with a variable allocation will regularly perform an evaluation which may increase or decrease the number of page frames of a process. When it comes to extra frames, these free frames will be taken. If none of these frames are available when a page fault occurs, the replacement algorithm selects a page from all the frames belonging to the process that caused the fault.

Variable allocation is also very useful when few processes are active. Finally, there is no reason to limit the resident set of a process. However, when many processes become active, the question arises how to determine the number of page frames dynamically. A key role in answering this question is played by the *active working set* $W(t, \Delta)$ of a process. This set contains, at time t , all pages requested by a process during the last Δ seconds that it was active (we sometimes use the notion of virtual time). The evolution of the size of this set is characterized by periods of stability that alternate with transition periods. During a transition, the active set will often increase sharply.

If we monitor the active set of all processes, we can regularly remove all pages that no longer belong to the active set of a process. Most systems don't really remove these pages, but use a page buffering mechanism to catch wrong decisions. This approach seems appealing, but has some difficulties. Like LRU, observing the past does not guarantee what is to come. In addition, tracking the set $W(t, \Delta)$ requires a lot of effort. In practice, one will therefore rather observe the page fault frequency (or rate). Many page faults indicate that the active working set is larger than the number of available frames.

A possible approach is to define a *use bit* (like the Clock algorithm) that we set to 1 when an active page is requested. For each page fault, we then check when the previous fault occurred. If it was very recent, it indicates an underestimation of the active set and the new page is added to the set of available pages. If the last fault occurred longer ago, the working set seems to be sufficiently large and all pages for which the use bit is still set to 0 are removed first. In this case we also reset all use bits to 0.

This approach seems effective, there is however a big disadvantage: in case of a transition period the number of used pages will grow strongly. This while there are also many pages in the set which are no longer used. This is not good for the stability of the system. To avoid this one can also evaluate the use bits after each Q page faults (in combination with two thresholds to guarantee a minimum and maximum time between evaluations).

As indicated, this mechanism will usually also use page buffering. Changes to the pages which are removed from the active set must also be made in secondary memory before they are actually removed from main memory. This can be done page by page, just before they are deleted (i.e. *demand cleaning*) or in batches (i.e. *pre-cleaning*). The former is less efficient, but only writes the necessary data to disk. A popular method is to write the contents of the page buffer to disk regularly.

4 Optimality of the MIN algorithm: a proof

In this section, we give an elementary proof of the optimality of the MIN algorithm. This proof was recently published, in April 2007, by Benjamin Van Roy in the journal Information Processing Letters. We consider a sequence of T page requests, $\omega_0, \dots, \omega_{T-1}$ and show that the MIN algorithm minimizes the number of page faults. Let Ω be the set of all pages and $S_t \subseteq \Omega$ the set of pages in memory at time t , i.e. just before request ω_t is placed. If $\omega_t \in S_t$, then $S_{t+1} = S_t$, otherwise ω_t is

added to S_t and another page must be removed from memory. The effort is to maximize $\sum_{t=0}^{T-1} 1(\omega_t \in S_t)$, where $1(A) = 1$ if A is true, otherwise $1(A) = 0$.

We start by defining $J_t(S_t)$ as the maximum number of hits remaining can occur from time t onwards. Consequently,

$$J_t(S_t) = 1(\omega_t \in S_t) + \max_{S_{t+1} \in U_t(S_t)} J_{t+1}(S_{t+1}), \quad (2)$$

for $t < T$ and $S_t \subseteq \Omega$, where $U_t(S_t)$ contains all possible subsets of Ω that may be in memory at time $t + 1$ (given that S_t was in memory at time t). In short,

$$U_t(S_t) = \{S \subseteq S_t \cup \omega_t : \omega_t \in S, |S| = |S_t|\}.$$

Furthermore, we note the following time when page ω is requested again becomes $\tau_t(\omega) = \min\{t \leq z < T : \omega_z = \omega\}$. When there is a page fault occurs at time t , the MIN algorithm will replace the page ω for which $\tau_t(\omega)$ is the largest (of all pages in memory). If a page ω is not called anymore after time t then $\tau_t(\omega) = \infty$.

To prove the optimality of the MIN algorithm we consider all bijective images h between two memory states S and S' (with $|S| = |S'|$). We then define d_t as the distance between S and S' :

$$d_t(S, S') = \min_{h \text{ bijection}} |\{\omega \in S : \tau_t(\omega) > \tau_t(h(\omega))\}|.$$

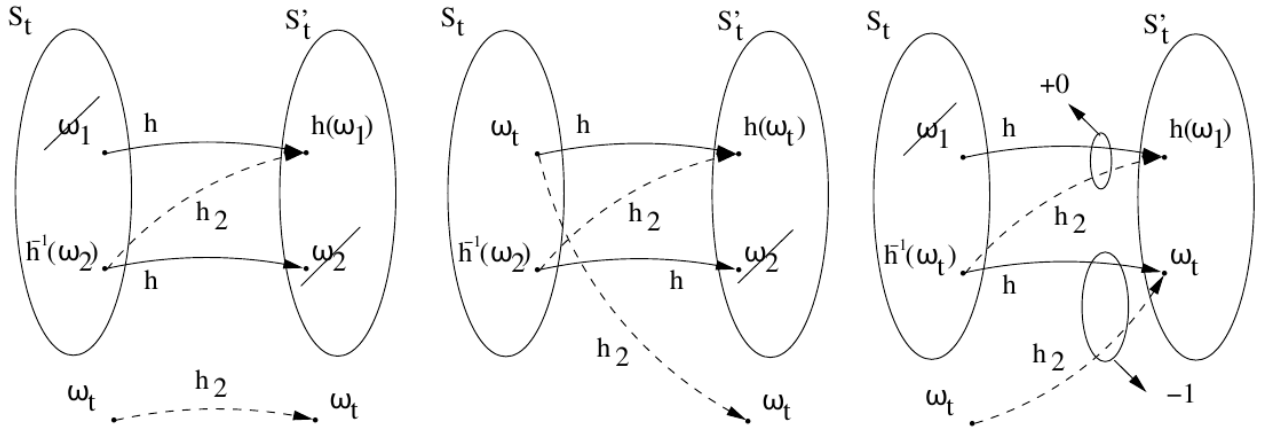


Fig. 36: Graphical representation of the inequalities

Let us consider the following example: $S = \{\omega_a, \omega_b, \omega_c\}$ and $S' = \{\omega_a, \omega_d, \omega_e\}$ with $\tau_t(\omega_a) = 8$, $\tau_t(\omega_b) = 13$, $\tau_t(\omega_c) = 6$, $\tau_t(\omega_d) = 11$, $\tau_t(\omega_e) = 4$. Then next bijection h :

$$h(\omega) = \begin{cases} \omega_a & \text{for } \omega = \omega_b, \\ \omega_d & \text{for } \omega = \omega_c, \\ \omega_e & \text{for } \omega = \omega_a, \end{cases}$$

ensure that $d_t(S, S')$ is equal to 1.

Now we prove the following lemma: $J_t(S') - J_t(S) \leq d_t(S, S')$ and this for $t \leq T$ and $S, S' \subseteq \Omega$ with $|S| = |S'|$. We adopt backward induction at t . $J_T(S) = J_T(S') = d_T(S, S') = 0$. Let $S_t = S$, $S'_t = S'$ and

let $S_{t+1} \in U_t(S_t)$ chosen by the MIN algorithm and $S'_{t+1} \in U_t(S'_t)$ by an optimal algorithm. Suppose the result holds for time $t + 1$: $J_{t+1}(S'_{t+1}) - J_{t+1}(S_{t+1}) \leq d_{t+1}(S_{t+1}, S'_{t+1})$. Then we find the following inequalities

$$d_{t+1}(S_{t+1}, S'_{t+1}) = \begin{cases} = d_t(S_t, S'_t) & \text{for } \omega_t \in S_t \cap S'_t, \\ \leq d_t(S_t, S'_t) & \text{for } \omega_t \in S_t \cup S'_t, \\ \leq d_t(S_t, S'_t) + 1 & \text{for } \omega_t \in S_t \setminus S'_t, \\ \leq d_t(S_t, S'_t) - 1 & \text{for } \omega_t \in S'_t \setminus S_t, \end{cases}$$

The equality is clear since $S_t = S_{t+1}$ and $S'_t = S'_{t+1}$ (and the h that minimizes $d_t(S_t, S'_t)$ can always be chosen such that $h(\omega_t) = \omega_t$, by mapping $h^{-1}(\omega_t)$ to $h(\omega_t)$ if this is not the case). For the other three cases we also refer to Figure 36. In the second case, ω_t is not part of either memory state. Suppose that $S_{t+1} = (S_t \cup \omega_t) \setminus \omega_1$ and $S'_{t+1} = (S'_t \cup \omega_t) \setminus \omega_2$. Let h be the bijection that realizes a minimum for $d_t(S_t, S'_t)$. Then we define h_2 identical to h except that $h_2(\omega_t) = \omega_t$ and $h_2(h^{-1}(\omega_2)) = h(\omega_1)$, then $d_t(S_t, S'_t)$ is maximally increased by one, namely when

$$\begin{aligned} \tau_t(\omega_1) &\leq \tau_t(h(\omega_1)), \\ \tau_t(h^{-1}(\omega_2)) &\leq \tau_t(\omega_2), \\ \tau_{t+1}(h^{-1}(\omega_2)) &> \tau_{t+1}(h(\omega_1)). \end{aligned}$$

However, given that the MIN algorithm chooses ω_1 , $\tau_t(\omega_1) \geq \tau_t(h^{-1}(\omega_2))$, which is together with the third equation gives a contradiction to the first equation. This proves the first inequality.

In the third case ω_t is part of S_t but not of S'_t . If we then define h_2 as h , but with $h_2(\omega_t) = \omega_t$ and $h_2(h^{-1}(\omega_2)) = h(\omega_t)$ then $d_t(S_t, S'_t)$ increases by one at most. In the latter case ω_t is part of S'_t , but not of S_t . Consequently $\tau_t(h^{-1}(\omega_t)) > \tau_t(\omega_t) = t$. We now choose h_2 identical to h , except that $h_2(\omega_t) = \omega_t$ and $h_2(h^{-1}(\omega_t)) = h(\omega_1)$. For the sake of the MIN algorithm, $\tau_t(\omega_1)$ must be $\geq \tau_t(h^{-1}(\omega_t))$, so if $\tau_t(\omega_1) \leq \tau_t(h(\omega_1))$ then it also holds that $\tau_t(h^{-1}(\omega_t)) \leq \tau_t(h(\omega_1))$, which decreases $d_t(S_t, S'_t)$ by at least one in total. It follows:

$$d_{t+1}(S_{t+1}, S'_{t+1}) \leq d_t(S_t, S'_t) + \mathbf{1}(\omega_t \in S_t) - \mathbf{1}(\omega_t \in S'_t).$$

Using this inequality, the induction hypothesis and equation (2) we then obtain

$$\begin{aligned} J_t(S'_t) &= \mathbf{1}(\omega_t \in S'_t) + J_{t+1}(S'_{t+1}) \\ &\leq \mathbf{1}(\omega_t \in S'_t) + d_{t+1}(S_{t+1}, S'_{t+1}) + J_{t+1}(S_{t+1}) \\ &\leq \mathbf{1}(\omega_t \in S_t) + d_t(S_t, S'_t) + J_{t+1}(S_{t+1}) \\ &\leq \mathbf{1}(\omega_t \in S_t) + \max_{\hat{S} \in U_t(S_t)} J_{t+1}(\hat{S}) + d_t(S_t, S'_t) \\ &= J_t(S_t) + d_t(S_t, S'_t), \end{aligned}$$

where the first equality follows from the optimality.

Given S_t , let S_{t+1} be the state we obtain by using the MIN algorithm and S_{t+1} be the state achieved by an optimal algorithm (also from S_t !). Since S_{t+1} minimizes the distance $d_{t+1}(\hat{S}_{t+1}, S'_{t+1})$ over all $\hat{S}_{t+1} \in U_t(S_t)$, which also includes S'_{t+1} , $d_{t+1}(S_{t+1}, S'_{t+1}) = 0$. On the other hand, we have $0 \leq J_{t+1}(S'_{t+1}) - J_{t+1}(S_{t+1}) \leq d_{t+1}(S_{t+1}, S'_{t+1})$, or still, $J_{t+1}(S_{t+1}) = J_{t+1}(S'_{t+1})$, which proves optimality.